

* Assignment :- 2 *

Q.1) Answer the following in short :

1. Explain the order by clause with Syntax and Example.

Ans The order by clause is used to sort the result-set in ascending or descending order.

→ The ORDER BY clause sorts the records in ascending order by default. To sort the record in descending order, use the DESC keyword.

→ Syntax:-

Select column_name(s) From table_name

ORDER BY column_name(s) ASC|DESC

→ Example:-

27.php

```
$mysql = mysqli_connect("localhost", "root", "", "db-demo") or die("error");
```

```
$q = "Select id, fname, lname From tbluser  
ORDER by lname";
```

```
$r = mysqli_query($mysql, $q);
```

28

2. Write down Syntax of SELECT, INSERT, UPDATE and DELETE.

Ans ^{Insert} ~~SELECT~~ Syntax:-

Insert into Table-name (Column1, Column2,)
Values (Value1, Value2, Value3 ...)

→ SELECT Syntax:-

Select Column-name-1, Column-name-2,
From Name-of-table.

→ UPDATE Syntax:-

Update table-name Set [Column-name1 = value1, ...
Column-N = value-N] where condition;

→ DELETE Syntax:-

DELETE From table-name where condition;

3. What is Flask? Is it open source framework?

Ans Flask is a web application framework written in Python.

→ It was developed by Armin Ronacher, who led a team of International Python enthusiasts called Pocco.

→ Flask is based on the Werkzeug WSGI toolkit and the Jinja2 Template engine. Both are Pocco

Projects-

→ Flask's framework is more explicit than Django's framework and is also easier to learn because it has less base code to implement a simple web-application.

→ Yes, Flask is an open-source Python web framework.

4. What is the default host name and port of flask?

Ans Host Name :- The default hostname is 127.0.0.1 i.e. localhost.

Port :- The Port number to which the server is listening to. The default Port number is 5000.

5. Why do we use Flask(--name--) in Flask?

Ans --name-- argument is passed in the Flask class to create its instance, which is then used to run the application.

→ Flask uses it to know from where to get resources, templates, static files, etc required to run the application.

6. Write the Significance of `url_for()` in flask.

Ans The '`url_for()`' function in Flask is essential because it dynamically creates URLs for routes, making it easy to generate links and maintain consistent URLs in your web application.

→ This function is useful in the sense that we can avoid hard-coding the URLs into the templates by dynamically building them using this function.

→ The first argument is the name of the specified function, and then we can pass any number of keyword argument corresponding to the variable part of the URL.

7. List out HTTP methods provided by Flask with its meaning.

<u>Ans</u> Method	Description
→ GET	It is the most common method which can be used to send data in the unencrypted form to the server.
→ HEAD	It is similar to the GET but used without the response body.
→ POST	It is used to send the form data to the server. The server does not cache the data transmitted using the Post method.

→ PUT It is used to replace all the current representation of the target resource with the uploaded content.

→ DELETE It is used to delete all the current representation of the target resource specified in the URL.

8. what WSGI?

Ans It is an acronym for web Server Gateway Interface which is a standard for Python web application development.

→ It is considered as the specification for the universal interface between the web server and web application.

→ WSGI is a specification that describes the communication between web servers and Python web applications or frameworks.

→ It explain how a web server communicates with Python web application/frameworks and application/frameworks can be chained for processing a request.

9. what do you understand by jinja delimiters?

Ans jinja2 template engine provides some delimiters which can be used in the HTML to make it capable of dynamic data representation.

- The Template System Provides some HTML Syntax which are Placeholders for variables and expressions that are replaced by their actual values when the template is rendered.
- The Jinja2 template engine Provides the following delimiters to escape from the HTML.

- `{ % ... % }` for Statements
- `{{ ... }}` for expressions to Print to the template output.
- `{# ... #}` for the comments that are not included in the template output.
- `# ... ##` for line Statements.

10. Write down the command to install flask framework of Python.

Ans We can now install the flask by using the following command.

- `$ pip install flask`

→ To Test the flask installation, open Python on the command line and type Python to open the Python Shell. Try to import the Package flask

→ `>>> import flask`
`>>>`

→ we have Successfully install the flask since we didn't get the error in the process.

Q.2 Do as Directed:

1) Explain with example how to Process Query in backend using Ajax.

Ans AJAX = Asynchronous javascript and XML.

→ AJAX is not a Programming language or is not a technology but group of inter-related technologies. AJAX technologies includes:

- HTML / XHTML and CSS
- DOM
- XML or JSON
- XMLHttpRequest
- JavaScript

→ AJAX allows web pages to be updated asynchronously by exchanging data with a web server ~~in~~ behind the scenes. This means that it is possible to update parts of a web page, without reloading the whole page.

→ You could perform Ajax calls using vanilla javascript as well as using ajax() method of jquery library.

→ The ajax() method is used to perform an Ajax (asynchronous HTTP) request.

→ Syntax:

`$.ajax({ name: value, name: value, ... })`

→ The Parameters specifies one or more name/value Pairs for the ajax request.

\$link = mysqli_connect("localhost", "root", "", "db_ajax") or die("Connection failed");

if(isset(\$_POST['query']))
\$q = "Select * from Student where
lastname like 'Y.%'";

\$result = mysqli_query(\$link, \$q);
if(mysqli_num_rows(\$result) > 0){
\$output = "<table border='1' cellpadding='10'>

<tr><th>First Name</th>
<th>Last Name</th>
<th>Contact No</th>

</tr>";

while(\$r = mysqli_fetch_row(\$result))
\$output .= "<tr>

<td>{\$r[1]}</td>

<td>{\$r[2]}</td>

<td>{\$r[3]}</td>

</tr>";

\$output .= "</table>";
echo \$output;
else
echo "No data";

??

Name	Description
data	Specifies data to be Send to the Server.
error	A function to run if the request fails.
Success	A function to be run when the request Succeeds.
type	Specifies the type of request. (GET or POST)
url	Specifies the URL to Send the request to. Default is the current Page

* Example's

```

-> <html>
    <head>
        <title> jQuery Auto Suggest Textbox using
        PHP Ajax </title>
        <Script src = "https://code.jquery.com/jquery-
        1.12.4.min.js"> </Script>
    </head>
    <body>
        <br />
        <br />
        <h3 align = "center"> jQuery Auto Suggest
        Textbox using PHP Ajax </h3>
    
```



```
</br>
```

```
</br>
```

```
</center>
```

```
<input type="text" id="Search" Placeholder="Search" Size="100" Style="padding: 20; border-color: blue;"/>
```

```
<p id="result"></p>
```

```
</center>
```

```
</body>
```

→ autocomplete PHP

```
<Script>
```

```
$(document).ready(function() {
```

```
//
```

```
$("#Search").keyup(function() {
```

```
var query = $("#Search").val();
```

```
$.ajax({
```

```
url: 'fetch.php',
```

```
method: 'POST',
```

```
data: {
```

```
query: query,
```

```
,
```

```
success: function(data) {
```

```
$("#result").html(data);
```

```
}
```

```
});
```

```
});
```

```
});
```

```
</Script>
```


→ fetch.php

<?php

```
$link = mysqli_connect("localhost", "root", "",
    "db-ajax") or die("Error");
```

```
if(isset($_POST['query'])) {
    $q = "Select * from student where
        lastname like '%".$_POST['query']."'";
```

```
$result = mysqli_query($link, $q);
if(mysqli_num_rows($result) > 0) {
    $output = "<table border=1 cellpadding=10";
```

```
<tr><th>First Name</th>
```

```
<th>Last Name</th>
```

```
<th>Contact No</th>
```

```
</tr>";
```

```
while($r = mysqli_fetch_row($result)) {
```

```
$output .= "<tr>
```

```
<td>$r[1]</td>
```

```
<td>$r[2]</td>
```

```
<td>$r[3]</td>
```

```
</tr>
```

```
";
```

```
}
```

```
$output .= "</table>";
```

```
echo $output;
```

```
}
```

```
else
```

```
echo "<h2>No data found...</h2>";
```

```
}
```

```
>>
```

2. Explain routing in flask with suitable examples

Ans APP routing is used to map the specific URL with associated function that is intended to perform some task. It is used to access some particular page in the web application.

- In other words, we can say that if we visit the particular URL mapped to some particular function, the output of that function is rendered on the browser's screen.
- Routing can be achieved by using `route()` and `add_url_rule()` functions.

1.) The `route()` function:

→ `from flask import *`
`app = Flask(__name__)`

```
@app.route('/home/<name>')
```

```
def home(name):
```

```
    return "<h2>hello, "+name+"</h2>"
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

- The converted can also be used in the URL to map the specified variable to the particular data type. For example, we can provide the integers or float like age or salary respectively.
- The following converters are used to convert the default string type to the associated data type:

- String: default
- int: used to convert the String to the integer
- float: used to convert the String to the float
- Path: It can accept the Slashes given in the URL.

→ Example:

```
from flask import *
app = Flask(__name__)
@app.route('/home/<int: age>')
def home(age):
    return "<h3> You are %d Years old" % age
```

```
if __name__ == "__main__":
    app.run(debug = True)
```

2) The add_url_rule() function:

→ There is one more approach to Perform Routing for the flask web application that can be done by using the add_url_rule() function of the Flask class.

→ The Syntax to use this function is given below.
add_url_rule(<url rule>, <endpoint>, <view function>)

→ This function is mainly used in the case if the view function is not given and we need to connect a view function to an endpoint externally by using this function.

→ Example :

```
from flask import *
app = Flask(__name__)
```

```
def about():
```

```
    return "<h2> This is about Page 2 / h2";
```

```
app.add_url_rule("/about", "about", about)
```

```
if __name__ == "__main__":
```

```
    app.run(debug = True)
```

3. How does file uploading works in Flask?
Explain with example.

Ans File uploading is the process of transmitting the binary or normal files to the server. Flask facilitates us to upload the files easily.

→ All we need to have an HTML form with the enctype set to multipart/form-data.

→ The server-side flask script fetches the file from the request object using request.files[] object. On successfully uploading the file, it is saved to the desired location on the server.

→ The uploaded file is saved to the temporary directory of the server for a while before it is saved to some desired location.

→ The name of the destination file can be obtained using the following syntax:-

```
name = request.files['file'].filename
```


- However, we can mention the path of the folder where the file is to be uploaded to the server and the maximum size of the uploaded file.
- This all can be done in the configuration settings of the flask object.

→ Syntax

`app.config['upload-FOLDER']` :- It is used to mention the upload folder.

`app.config['MAX-CONTENT-PATH']` :- It is used to mention the maximum size of the file to be uploaded.

→ Example's

→ file-upload-form.html

`<html>`

`</head>`

`<title>upload </title>`

`</body>`

`<form action = "/Success" method = "Post" enctype = "multipart/form-data">`

Select Any file:

`<input type = "file" name = "file" />`

`<input type = "Submit" value = "upload" />`

`</form>`

`</body>`

`</html>`

→ Success.html

```

<html>
<head>
  <title> Success </title>
</head>
<body>
  <p> File uploaded Successfully </p>
  <p> File name: {{name}} </p>
</body>
</html>

```

→ upload.py

```

from flask import *
app = Flask(__name__)
@app.route('/')
def upload():
    return render_template("file_upload_form.html")

@app.route('/success', methods = ['POST'])
def success():
    if request.method == 'POST':
        f = request.files['file']
        f.save(f.filename)
        return render_template("Success.html",
                                name = f.filename)

```

```

if __name__ == '__main__':
    app.run(debug = True)

```


4. Explain with example how to get query string in Flask

Ans Get Method is used to get query string in flask.

→ we can specify form method in the route() as:
`@app.route('/login', methods = ['GET'])`

→ To retrieve the value use following syntax:
`uname = request.args.get('uname')`

→ Example's

→ Login.html

`<html>`

`<body>`

`<form action = "http://localhost:5000/login" method = "get">`

`<table>`

`<tr><td>Name</td>`

`<td><input type = "text" name = "uname"></td></tr>`

`<tr><td>Password</td>`

`<td><input type = "password" name = "pass"></td></tr>`

`</tr>`

`<tr><td><input type = "submit"></td></tr>`

`</table>`

`</form>`

`</body>`

`</html>`

→ get_example.py

```
from flask import *  
app = Flask(__name__)
```

```
@app.route('/login', methods = ['GET'])  
def login():
```

```
    uname = request.args.get('uname')
```

```
    password = request.args.get('pass')
```

```
    if uname == "tmt" and password == "Password":
```

```
        return "welcome %s" % uname
```

```
    else:
```

```
        return "Invalid user!!!"
```

```
if __name__ == '__main__':
```

```
    app.run(debug = True)
```

5. Explain HTTP POST method in Flask with example.

Ans → POST Method:

→ To handle the post requests at the server, let us first create a form to get some data at the client side from the user, and we will try to access this data on the server by using the POST request.

→ we can specify form method in the route() as:
`@app.route('/login', methods = ['POST'])`

→ To receive the values use following syntax:

uname = request.form['uname']

→ Example:

Login.html

<html>

<body>

<form action = "http://localhost:5000/login"
method = "Post">

<table>

<tr><td>Name</td>

<td><input type = "text" name = "uname"></td>
</tr>

<tr><td>Password</td>

<td><input type = "Password" name = "pass"></td>
</tr>

<tr><td><input type = "submit"></td></tr>

</table>

</form>

</body>

</html>

→ POST example.py

from flask import *

app = Flask(__name__)

@app.route('/login', methods = ['POST'])

def login():

uname = request.form['uname']

password = request.form['pass']

```
if uname == "tmt" and Password == "Password":  
    return "Welcome %s" % uname  
else:  
    return "Invalid user!!!"
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

6. How we can embed Python Statement in HTML?
Explain giving Suitable example.

Ans As we have seen in the file Structure in Previous Section, for working with the HTML files we need to consider the Structure we have seen earlier.

→ So we will keep all the html files within the same directory say "templates". Our data will be flow from the Python to these html files.

→ Once we are able to start and run flask server, we can start sending inputs from the Python backend, and later on, we will see how inputs from web UI flow to the backend processing and render that result on the website.

→ The main idea of using a templating engine in web development is to render data dynamically in HTML files.

→ To represent such data forms in HTML files, there are 3 types of Syntaxes are available:

1. Expressions:- These are the Simple Variable representation that is used for displaying any data type from the python backend.
- It is similar to Print Statement in Programming languages.
 - The Syntax is `{{...}}`

2. Statements:- These Syntaxes are used to implement control structures such as for loop, if-else, macros, blocks, imports, and much more things.
- The Syntax is `{%...%}`

3. Comments: One can also add Jinja comments to the template codes. These are represented as `{#...#}`.

→ Example:

Script.py

```
from flask import *  
app = Flask(__name__)
```

```
@app.route('/table/<int:num>')
```

```
def table(num):
```

```
    return render_template('Print-table.html',  
                           n=num)
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

→ Print - table.html

<html>

<head>

<title>Print table</title>

</head>

<body>

<h2>Printing table of 5</h2>

<table>

<tr>

<td>

</td>

</tr>

</table>

→ output:

127.0.0.1:8000/table/5

Printing table of 5

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

5 x 6 = 30

5 x 7 = 35

5 x 8 = 40

5 x 9 = 45

5 x 10 = 50.

* Assignment :- 3 *

Q.1 Answer the following in short:

1. What is difference between `exists()` and `isfile()` in Python?

| | <u>Ans</u> <code>exists()</code> | <code>isfile()</code> |
|----|--|---|
| 1. | checks if a path (file or directory) exists or not. | checks if a path exists and is a regular file. |
| 2. | <code>os.Path.exists()</code> | <code>os.Path.isfile()</code> |
| 3. | True if Path exists, False if not exists. | True if Path is a regular file and exists, False if not or if it's a directory or another type of object. |
| 4. | <p>Example:</p> <pre>os.Path.exists('C:\\xampp\\htdocs\\scripts\\unit4\\remove.py')</pre> <p>Returns True.</p> | <p>Example:</p> <pre>os.Path.isfile('C:\\xampp\\htdocs\\scripts\\unit4\\remove.py')</pre> <p>Returns True</p> |

2. what is the use of split() function in python?

Ans The split method will split a path into a tuple that contains the directory and the file.

→ The split() method took the path and split it in such way that the last sub-folder become the second element of the tuple with rest of the path in the first element.

→ >>> dirname, fname = os.path.split('c:\\xampp\\htdocs\\Scripts\\Unit 4\\remove.py')

>>> print(dirname)

c:\\xampp\\htdocs\\Scripts\\Unit 4

>>> print(fname)

remove.py

3. Name the function that will return just the filename of a path. Give one example

Ans The basename function will return just the filename of a path.

→ os.path.basename() method in python is used to get the base name in specified path.

→ os.path.basename() method returns the tail part after splitting the specified path into (head, tail) parts.

→ Example:

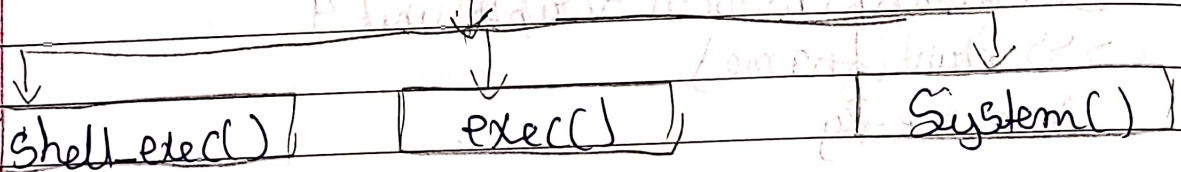
Page No.
 Date
 >>> import os
>>> os.path.basename('C:\\xampp\\htdocs\\
Scripts\\unit 4\\remove.py')
Output:-
'remove.py'

Q.2 Do as directed:-

1. Explain with example how to execute Python Script in PHP.

Ans

ways to execute
py file



1) shell_exec() method:-

- It is used to execute the commands via shell and return the complete output as a String.
- The shell_exec() is an alias for the backtick operator.
- Syntax:
Shell_exec(\$cmd)

2) exec() method:-

- The exec() is used to execute an external Program and returns the last line of the

output. It also returns NULL if no command run properly.

→ Syntax:

```
exec(String $command, array $input = null,
      int $result_code = null): String | false
```

3) System():

→ It is for executing a system command and immediately display the output - Presumably text.

→ It returns the last line of the command output on success, and false on failure.

→ Syntax:

```
System(String $command, int $result_code = null):
String | false
```

→ Example:

```
hello.py
Printf("Hello from Python!!");
Print("I am eager to know how this works!!!")
```

2) php

```
$result = `python hello.py`;
echo $result.
```

```
$Str-output = shell_exec('python hello.py');
echo $Str-output;
```

```
exec('python hello.py', $lines);
foreach ($lines as $l)
    echo "$l";
```


Page No.
 Date

```
echo "hello";  
System('python hello.py');
```

2. Explain any 5 functions of Python OS module with example.

Ans Functions of OS Module

- 1) `os.chdir()`
- 2) `os.getcwd()`
- 3) `os.mkdir()`
- 4) `os.makedirs()`
- 5) `os.remove()`

1) `os.chdir()`

→ The `os.chdir` function allows us to change the directory that we are currently running our Python session in.

2) `os.getcwd()`:

→ If you want to actually know what path you are currently in, then you would call `os.getcwd()`.

* Example 5

```
>>> import os  
>>> os.getcwd()  
: 'C:\\xampp\\htdocs\\scr\\WS\\unit 4'
```

```
>>> os.chdir("c:\\xampp\\htdocs\\Scripts")
>>> os.getcwd()
'c:\\xampp\\htdocs\\Scripts'
```

3) os.mkdir()

- These are used for creating directories.
- The first one is os.mkdir(), which allows us to create a single folder.

4) os.makedirs()

- The os.makedirs() function will create all the intermediate folders in a path if they don't already exist.
- Basically this means that you can create a path that has nested folder in it.

* Example:-

```
>>> os.mkdir('Unit 5')
>>> path = 'c:\\xampp\\htdocs\\2023\\july\\15'
>>> os.makedirs(path)
```

5) os.remove()

- The os.remove() functions are used for deleting files and directories respectively.
- Python remove() method removes the file path. If the path is directory, OSError is raised.

→ Syntax:
`os.remove(Path)`

→ Example's

```
import os
Print("The dir is: ", os.listdir(os.getcwd()))
os.remove("temp1.txt")
Print("The dir after removal of Path:",
      os.listdir(os.getcwd()))
```

Output's

c:\xampp\htdocs\Scripts\unit4 > python remove.py

3) Explain any 3 functions of subprocess module of Python.

Ans A running Program is called a Process.
 → Each Process has its own System State, which includes memory, lists of open files, a Program counter that keeps track of the instruction being executed, and a call stack used to hold the local variable of functions.

→ A Program can create new Processes using library functions such as those found in the os or subprocess modules such `os.fork()`, `subprocess.Popen()`, etc. However, these Process, known as Subprocess.

α ~~Recall~~ The 3 function of Sub-module

- 1) call()
- 2) Popen()
- 3) decode()

1) call() function:-

- The Subprocess module provides a function named call.
- This function allows you to call another program, wait for the command to complete and then return the return code.
- It accepts one or more arguments as well as the following keyword arguments (with their defaults): `stdin=None`, `stdout=None`, `stderr=None`, `shell=False`.
- Syntax:

```
subprocess.call(args, *, stdin=None, stdout=None,  
                stderr=None, shell=False)
```

→ Example:

```
>>> import subprocess  
>>> subprocess.call("notepad.exe")  
0
```

- If you run this on windows machine, you should see Notepad open up. You will notice that IDLE waits for you to close Notepad and

then it returns a code `zero(0)`. This means that it completed successfully. If you receive anything except for a zero, then it usually means you have had some kind of error.

2) Popen class

→ The `Popen` class executes a child program in a new process. Unlike the `call` method, it does not wait for the called process to end unless you tell it to using by using the `wait` method.

→ It is like OS System command. Python method `Popen()` opens a pipe to or from command.

→ The return value is an open file object connected to the pipe, which can be read or written depending on whether mode is 'r' (default) or 'w'.

→ Syntax:

OS: `Popen(command[, mode])`

→ Example:

~~→~~

`import subprocess`

~~→~~ `prog`

`cm = 'dir'`

`p1 = subprocess.Popen(cm, shell=True)`

`p1.wait()`

```
if p1.returncode == 0:
    print("Successfully run")
else:
    print(p1.stderr)
```

3) decode():

- This method is used to convert from one encoding scheme, in which argument string is encoded to the desired encoding scheme.
- This works opposite to the encode. It accepts the encoding of the encoding string to decode it and returns the original string.
- Syntax:


```
decode(encoding, error)
```

α Parameters:-

- encoding :- Specifies the encoding on the basis of which decoding has to be performed.
- error :- Decides how to handle the errors if they occur, e.g 'Strict' raises Unicode error in case of exception and 'ignore' ignores the errors occurred.

α Example:-

```
import subprocess
str = "Python"
str_encoded = str.encode('utf-16', 'Strict')
```



```
Print("The encoded string is: ", Str-Encoded)
Str-decoded = Str-Encoded.decode('utf-16', 'strict')
Print("The decoded string is: ", Str-decoded)
```